Memoria Caché: práctica con RIPES

Simulación de parámetros de caché y análisis de costo de hardware

---

Simulador:	RIPES v2.2.x — modo `Single-Cycle RV32IM`
Archivos:	carpeta `asm/` del repositorio
Compilador:	integrado en RIPES — pegar código C en la pestaña `C` del editor

---

0 Configuración del compilador C en RIPES

RIPES incluye soporte para compilar código C directamente desde el editor. Antes del primer uso, configurar el compilador una sola vez:

Configuración RIPES

Opción A — Script automático (recomendado, ejecutar una sola vez):

```
chmod +x scripts/setup_compiler.sh
./scripts/setup_compiler.sh
```

Reiniciar RIPES después de ejecutar el script.

Opción B — Configuración manual desde la GUI de RIPES:

1. Ir a `Edit` → `Settings` (o ícono de engranaje).
2. Seleccionar la pestaña `Compiler`.
3. Completar los campos:

Compiler path:	`/usr/bin/riscv64-unknown-elf-gcc`
Compiler args:	`-O1`
Linker args:	`-nostdlib -nostartfiles`

4. Hacer clic en `OK` y reiniciar RIPES.

Por qué estos valores:

- `riscv64-unknown-elf-gcc` es el único compilador del sistema que tiene soporte para `rv32im/ilp32` (multilib completo). RIPES agrega `-march=rv32im -mabi=ilp32` automáticamente según el procesador seleccionado; no es necesario repetirlos en Compiler args.
- `-nostdlib -nostartfiles` evita que el linker busque la libc de newlib (no instalada para rv32). RIPES no la necesita: arranca directamente desde `main()`.

0 Objetivos

- Medir en RIPES cómo cambia el miss rate al variar individualmente el **tamaño total**, el tamaño de línea y la asociatividad de la caché.
- Relacionar los patrones de acceso del código (C y assembly) con los miss rates observados.
- Calcular el costo de hardware (bits de almacenamiento, comparadores, estado LRU) en función de la asociatividad.
- Encontrar analíticamente la asociatividad que minimiza el AMAT considerando el impacto del hit time en el hardware real.

Referencia rápida

Parámetros de la caché y nomenclatura:

$$b = \log_2 B, \quad s = \log_2 S, \quad t = 32 - s - b$$

$$\text{AMAT} = t_h + \text{MR} \times t_m$$

B = tamaño de bloque (bytes)	b = bits de offset
S = número de sets	s = bits de índice
W = asociatividad (vías)	t = bits de tag
t_h = hit time (ciclos)	t_m = penalidad de miss (ciclos)

1 Parte 0B — Assembly en RIPES: acceso a arreglos

Nota

Archivo: `ej_array.s` — cargar con `File` → `Load assembly file`.

Implementa `C[i] = A[i] + B[i]` con paso configurable en assembly RV32IM. El `STEP` se controla modificando el inmediato de `li t6, ?` en la etiqueta `compute:`.

1.1 Actividad 1 — Identificación de accesos a memoria en el assembler

Actividad

Abrir `ej_array.s` y localizar el loop etiquetado `loop:`. Identificar las instrucciones de acceso a datos (`lw`/`sw`):

Acceso	**Instrucción en ej_array.s**	**Tipo**
Leer `A[i]`		load
Leer `B[i]`		load
Escribir `C[i]`		store

1.2 Actividad 2 — Perfilado con acceso secuencial (STEP = 1)

Atención

Siempre hacer Reset antes de cada corrida. Sin Reset, los bloques de corridas anteriores permanecen en caché y contaminan las estadísticas.

Actividad

Caché: L1 Data Cache habilitada, Line size = **4 words** (16 B), LRU. Verificar que `li t6, 1` (STEP=1) esté en `ej_array.s`.

Configuración	Accesos	Hits	Misses	Miss rate
Direct-Mapped (Sets=16, Ways=1)				
2-Way Set Assoc. (Sets=8, Ways=2)				
4-Way Set Assoc. (Sets=4, Ways=4)				

1. ¿Qué set tiene mayor actividad en la fase de suma? ¿Qué tags hay cargados en ese set al finalizar?
2. ¿En qué configuración los tres arrays (A, B, C) coexisten sin conflicto?

Usando los miss rates medidos, calcular el AMAT ($t_h = 1$ ciclo, $t_m = 100$ ciclos):

Configuración	MR	AMAT (ciclos)	Factor vs. DM
Direct-Mapped			1.0×
2-Way			
4-Way			

Checkpoint

En 4-Way con STEP=1: ¿a partir de qué iteración el set del conflicto queda estable? ¿Cuántos misses compulsorios son inevitables sin importar la asociatividad?

1.3 Actividad 3 — Impacto del stride[1] de acceso

Nota

Modificar el stride cambiando `li t6, ?` en `ej_array.s`:

- `li t6, 1` → STEP=1: 1 miss / 4 accesos con bloque 16 B
- `li t6, 4` → STEP=4: 1 miss / acceso (bloque 16 B)
- `li t6, 8` → STEP=8: miss garantizado, además salta bloques

Actividad

Antes de medir, calcular teóricamente el miss rate para la config **4-Way, Sets=4, Line=4 words (16 B)**, con N=64 elementos y 3 arrays:

STEP	Accesos totales	Misses esperados	MR teórico	Motivo
1				
4				
8				

Medición en RIPES (Reset entre cada corrida):

STEP	Accesos	Hits	Misses	MR medido	MR teórico
1					
4					
8					

Variante (STEP=4 fijo, variar tamaño de bloque):

Corrida	Line size	Sets	Ways	Miss rate
A	1 word (4 B)	16	4	
B	2 words (8 B)	8	4	
C	4 words (16 B)	4	4	
D	8 words (32 B)	2	4	

[1] *Stride*: distancia (en elementos) entre accesos consecutivos a memoria. Un stride de 1 accede a posiciones contiguas; un stride mayor salta posiciones, reduciendo la localidad espacial.

Checkpoint

1. ¿A partir de qué STEP el miss rate llega al 100 %?
2. Con STEP=4 y bloque 16 B: ¿qué porcentaje de los bytes traídos se usan? ¿Cómo se llama este fenómeno?
3. ¿A partir de qué tamaño de bloque el MR baja con STEP=4?

2 Parte I — Simulación en RIPES: barrido de parámetros de caché

Configuración RIPES

Flujo de trabajo en RIPES con código C:

1. Abrir RIPES y seleccionar procesador `Single-Cycle RV32IM`.
2. En el panel izquierdo, hacer clic en la pestaña `C` del editor. Pegar el código C del ejercicio. RIPES compila y carga automáticamente.
3. Para cambiar un parámetro (`#define N`, `#define STEP`, etc.), editar el valor directamente en el editor C y presionar `Run` o `Step`. RIPES recompila al instante.
4. En la barra lateral, activar `Cache` → `Data Cache`.
5. Ajustar `Ways`, `Lines` y `Words/Line` según cada ejercicio. La capacidad total es:

$$\text{Capacidad} = \text{Ways} \times \text{Lines} \times \text{Words/Line} \times 4\ \text{B}$$

6. Hacer **Reset** antes de cada corrida.
7. Leer `Accesses`, `Hits`, `Misses` y `Miss rate` en el panel de caché.

Nota: los archivos `.s` de la carpeta `asm/` se cargan con `File` → `Load assembly file` cuando el ejercicio lo especifique.

Atención

Siempre hacer Reset antes de cada corrida. Sin Reset, los bloques de corridas anteriores permanecen en caché y contaminan las estadísticas. El estado inicial debe ser caché vacía (*cold start*).

2.1 Ejercicio 1 — Efecto del tamaño total de caché

Concepto: cuando el working set supera la capacidad de la caché, los bloques que ya estuvieron son desalojados antes de reutilizarse (*capacity misses*). Al aumentar el tamaño de la caché, llega un punto en que el working set cabe completamente y los misses en la segunda pasada desaparecen.

Código C: pegar el siguiente programa en el editor C de RIPES. Para cambiar el tamaño del array, modificar `#define N` directamente en el editor.

```c
#define N 128    /* modificar: 32, 64, 128, 256 */

static int A[N];

int main(void)
{
    int i, s1 = 0, s2 = 0;

    /* inicializacion: primera escritura a cada elemento */
    for (i = 0; i < N; i++)
        A[i] = i + 1;

    /* Primera pasada: misses compulsorios puros */
    for (i = 0; i < N; i++)
        s1 += A[i];

    /* Segunda pasada: hits si A cabe en cache, misses de capacidad
       si no */
    for (i = 0; i < N; i++)
        s2 += A[i];

    return (s1 + s2) & 0xFF;
}
```

Listing 1: cache_size_test.c — pegar en el editor C de RIPES

Ejercicio

Parámetros fijos: Direct-Mapped (`Ways=1`), `Words/Line=4` (bloque 16 B), política LRU.

Pegar el código con `N=128`. El working set del array es $128 \times 4 = 512$ B. Variar el tamaño de la caché modificando `Lines`:

Configuración	**Capacidad**	**Accesos tot.**	**Misses**	**MR (%)**	**AMAT** (t_m=100)
Ways=1, Lines=4	64 B				
Ways=1, Lines=8	128 B				
Ways=1, Lines=16	256 B				
Ways=1, Lines=32	512 B				
Ways=1, Lines=64	1 KB				

1. ¿En qué configuración el miss rate cae drásticamente? Justificar comparando la capacidad de la caché con el tamaño del working set (512 B de datos + código de inicialización).
2. ¿Cuántos misses espera en la primera pasada vs. la segunda pasada? (Usar el panel `Step` para separar las dos fases.)
3. ¿A qué tipo de miss corresponden los misses de la primera pasada? ¿Y los de la segunda cuando el array no cabe?

Conclusión

El miss rate cae a su valor mínimo (solo misses compulsorios de la primera pasada) cuando la capacidad de la caché iguala o supera el working set. Duplicar la caché más allá de ese punto no reduce el miss rate.

Variante: en el editor C de RIPES, cambiar `#define N` a 32, 64 y 256. RIPES recompila automáticamente. Repetir con la configuración `Lines=16` (256 B) y registrar cómo cambia el miss rate de la segunda pasada.

N (elementos)	**Working set**	**¿Cabe en 256 B?**	**MR 2ª pasada**
32	128 B		
64	256 B		
128	512 B		
256	1024 B		

2.2 Ejercicio 2 — Efecto del tamaño de línea (cache line size)

Concepto: una línea más grande trae más datos contiguos por miss. Si el stride de acceso es menor que el tamaño de línea, los datos traídos se usan (↑ eficiencia). Si el stride supera el tamaño de línea, cada acceso cae en un bloque diferente y el overhead de miss se repite sin beneficio.

Código C: pegar en el editor C de RIPES y modificar `#define STEP` directamente para cada experimento.

```c
#define N     128    /* elementos por array */
#define STEP    4    /* stride: cambiar a 1, 4, 8 segun el
    experimento */

static int A[N], B[N], C[N];

int main(void)
{
    int i, suma = 0;

    /* inicializacion secuencial */
    for (i = 0; i < N; i++) {
        A[i] = i + 1;
        B[i] = N - i;
    }

    /* kernel con stride STEP: C[i] = A[i] + B[i] */
    for (i = 0; i < N; i += STEP) {
        C[i] = A[i] + B[i];
        suma += C[i];
    }

    return suma & 0xFF;
}
```

Listing 2: stride_test.c — pegar en el editor C de RIPES

Ejercicio

Parámetros fijos: capacidad total 256 B, `Ways=4` (4-way), política LRU, `STEP=4` (stride = 16 bytes entre accesos a cada array).

Variar `Words/Line` para cambiar el tamaño de línea.

Words/Line	**Bloque**	**Lines** (Sets×Ways)	**Accesos**	**MR (%)**	**AMAT** (t_m=100)
$2^0 = 1$ word	4 B	$64 \times 1 =$			
$2^1 = 2$ words	8 B				
$2^2 = 4$ words	16 B				
$2^3 = 8$ words	32 B				

1. Con `STEP=4` y bloque de 16 B: el stride en bytes es $4 \times 4 = 16$ B. ¿Cuántos elementos del bloque cargado se usan?
2. Con bloque de 32 B: el stride (16 B) es menor que el bloque. ¿Cuántos accesos de stride=4 comparten ahora el mismo bloque? ¿Cómo cambia el miss rate?
3. ¿Existe un tamaño de línea para el cual el miss rate es mínimo? ¿Podría ocurrir que una línea demasiado grande empeore el miss rate? ¿Bajo qué condición?

Extensión: en el editor C de RIPES, cambiar `#define STEP` a 1 y a 8. Construir la tabla de miss rate para las combinaciones stride × tamaño de línea:

	Tamaño de línea (Words/Line)			
STEP	1 word (4 B)	2 words (8 B)	4 words (16 B)	8 words (32 B)
1				
4				
8				

Conclusión

El miss rate mínimo con stride S se obtiene cuando el bloque contiene al menos S elementos, es decir cuando $B \geq S \times 4$ bytes. Para strides grandes, aumentar el bloque no ayuda si el patrón de acceso nunca reutiliza los datos extra traídos.

2.3 Ejercicio 3 — Efecto de la asociatividad

Concepto: los *conflict misses* ocurren cuando múltiples bloques populares compiten por el mismo set. Al aumentar la asociatividad (más vías por set), más bloques conflictivos pueden coexistir. Existe un umbral: cuando $W \geq N_c$ (número de bloques en conflicto), los conflict misses desaparecen.

Archivo: `ej_conflictos.s` (4 accesos que mapean al mismo set, repetidos dos veces).

Ejercicio

Parámetros fijos: capacidad 256 B, bloque 16 B (`Words/Line=4`), política LRU.

Cargar `ej_conflictos.s` y ejecutar con distintas asociatividades. El programa realiza 8 accesos en total: A, B, C, D (primera ronda) y A, B, C, D nuevamente (segunda ronda), donde A/B/C/D están separadas por 256 B (= capacidad total de la caché).

Configuración	**Ways**	**Sets**	**Accesos**	**Hits**	**MR (%)**	**AMAT** (t_m=100)
Direct-Mapped	1	16				
2-Way	2	8				
4-Way	4	4				
8-Way	8	2				

1. Simular en pizarra (sin RIPES) el comportamiento de la caché DM y 4-Way con la secuencia A, B, C, D, A, B, C, D. ¿Cuántos misses ocurren en cada caso?
2. ¿Por qué 2-Way tiene el mismo miss rate que DM en este programa?
3. ¿A partir de qué asociatividad los conflict misses desaparecen? Enunciar la regla general.
4. Con 8-Way: ¿el miss rate es menor que con 4-Way? ¿Cuántos misses son inevitables aunque la asociatividad sea infinita?

Extensión con código C: pegar el siguiente esqueleto en el editor C de RIPES. Cambiar `#define K` directamente en el editor para variar el número de bloques conflictivos. El programa replica el patrón de `ej_conflictos.s` desde C.

```c
#define CACHE_SIZE  256 /* bytes: ajustar config de RIPES */
#define BLOCK_SIZE   16 /* bytes */
#define K             4 /* bloques en conflicto: 2, 4, 6, 8 */
#define GAP  (CACHE_SIZE / (int)sizeof(int))   /* separacion */
#define N     4 /* accesos por bloque (1 por word del bloque) */

/* K arrays separados por GAP enteros: todos mapean al mismo set */
static int banks[K][GAP + N];

int main(void) {
    int k, i, s = 0;
    /* inicializar */
    for (k = 0; k < K; k++)
        for (i = 0; i < N; i++)
            banks[k][i] = k * N + i;
    /* patron conflictivo: dos pasadas */
    for (int pass = 0; pass < 2; pass++)
        for (k = 0; k < K; k++)
            /* acceso al primer elemento de cada banco */
            s += banks[k][0];
    return s & 0xFF;
}
```

Listing 3: Pegar en editor C de RIPES — cambiar K = 2, 4, 6, 8

Cambiar `#define K` en el editor C de RIPES a 2, 4, 6 y 8 (RIPES recompila automáticamente). Registrar el miss rate para DM y 4-Way:

K (bloques conflictivos)	**MR — DM**	**MR — 4-Way**
2		
4		
6		
8		

2.4 Ejercicio 4 — Acceso a matrices en C: row-major vs. col-major

Concepto: el orden de recorrido de una matriz bidimensional puede multiplicar el miss rate por un factor de N (dimensión de la matriz), aunque la operación realizada sea exactamente la misma.

Código C: pegar en el editor C de RIPES. Para variar la dimensión, cambiar `#define N` directamente en el editor.

```c
#define N 8   /* dimension de la matriz NxN; cambiar a 16 o 32 */

static int M[N][N];

static void fill(void)
{
    int i, j;
    for (i = 0; i < N; i++)
        for (j = 0; j < N; j++)
            M[i][j] = i * N + j + 1;
}

/* Fila por fila: stride = 4 B (localidad espacial alta) */
static int row_sum(void)
{
    int i, j, s = 0;
    for (i = 0; i < N; i++)
        for (j = 0; j < N; j++)
            s += M[i][j];
    return s;
}

/* Columna por columna: stride = N*4 B (localidad espacial baja) */
static int col_sum(void)
{
    int i, j, s = 0;
    for (j = 0; j < N; j++)
        for (i = 0; i < N; i++)
            s += M[i][j];
    return s;
}

int main(void)
{
    fill();
    int r = row_sum();
    int c = col_sum();
    return r - c;
}
```

Listing 4: matrix_row_col.c — pegar en editor C de RIPES

Ejercicio

Parámetros fijos: 256 B, 4-Way, bloque 16 B, LRU. Con `N=8`: la matriz ocupa $8 \times 8 \times 4 = 256$ B exactamente.

Función	Accesos	Misses	MR (%)	Stride entre accesos
`fill()`				4 B (secuencial)
`row_sum()`				4 B (secuencial)
`col_sum()`				$N \times 4$ B

1. Calcular analíticamente el stride de `col_sum()` con `N=8`: entre dos accesos consecutivos, ¿cuántos bytes se saltan?
2. ¿Cuántos elementos del bloque de 16 B cargado en `col_sum()` se utilizan efectivamente antes de que el bloque sea desalojado?
3. En el editor C de RIPES, cambiar `#define N 8` a `#define N 16`. La matriz ahora ocupa $16 \times 16 \times 4 = 1024$ B. ¿Cómo cambia el MR de `row_sum()`? ¿Por qué?

N	Tamaño matriz	MR row_sum	MR col_sum
8	256 B		
16	1024 B		
32	4096 B		

Conclusión

Para `row_sum()`, el miss rate permanece bajo independientemente de N (acceso secuencial, localidad espacial alta). Para `col_sum()`, el miss rate depende de cuántos elementos de cada columna caben en caché: cuando N supera el número de líneas disponibles, el MR llega al 100 %.

3 Parte II — Simulación RTL del hardware de la caché

Los ejercicios anteriores mostraron que aumentar la asociatividad reduce los conflict misses. Esta parte usa **herramientas de diseño de hardware real** para cuantificar ese costo. El flujo es:

1. **Verilog paramétrico** — `cache_comparator.v` y `plru_state.v` modelan la lógica del comparador de tag, el MUX de datos y el estado PLRU. El parámetro `W` controla la asociatividad.
2. **Simulación funcional** con `iverilog` — verifica que la lógica de hit/miss sea correcta antes de sintetizar.
3. **Síntesis lógica** con **Yosys** — traduce el Verilog a celdas genéricas y reporta el número de puertas, flip-flops y bits de señal para cada W.

Configuración RIPES

Configuración del entorno (carpeta `hw/`):

```
# Instalar Yosys (una sola vez)
sudo apt install yosys

# Verificar el Verilog con simulacion funcional
make sim

# Sintetizar para W = 1, 2, 4, 8, 16 y guardar resultados
make synth

# Flujo completo
make all
```

Cada vía adicional agrega hardware: comparadores de tag en paralelo, un MUX más ancho para seleccionar el dato, y más bits de estado para LRU. Este overhead no es gratuito: ocupa área y aumenta el hit time.

Referencia rápida

Fórmulas de recursos de hardware. Sea: capacidad total C, bloque B bytes, W vías, dirección A bits.

$$S = \frac{C}{B \times W} \qquad \text{(número de sets)}$$

$$b = \log_2 B, \quad s = \log_2 S, \quad t = A - s - b \qquad \text{(campos de la dirección)}$$

$$\text{Bits de datos} = C \times 8 \qquad \text{(fijo, independiente de } W\text{)}$$

$$\text{Bits de tag+valid} = \frac{C}{B} \times (1 + t) \qquad \text{(}C/B\text{ líneas en total)}$$

$$\text{Comparadores} = W \text{ comparadores de } t \text{ bits c/u} \qquad \text{(operan en paralelo por set)}$$

$$\text{Bits de LRU (PLRU)} = S \times (W - 1) \qquad \text{(árbol pseudo-LRU)}$$

$$\text{MUX de vía} = W\text{:1 sobre } B \times 8 \text{ bits} \qquad \text{(selecciona la vía ganadora)}$$

3.1 Ejercicio 5 — Simulación funcional: verificar la lógica de hit

Ejercicio

Paso 1 — Ejecutar la simulación funcional.

```
cd hw/ && make sim
```

Deben aparecer 5 líneas "`PASS`". Si alguna falla, indica un bug en el diseño o en el testbench.

Paso 2 — Agregar un caso de prueba propio.
Editar `tb_cache.v` y agregar un test 6 donde ambas vías tienen el valid bit en 1 pero solo una tiene el tag correcto. ¿Cuál es el resultado esperado de `hit` y `data_out`?

Paso 3 — Leer el módulo Verilog. Abrir `hw/cache_comparator.v`, identificarl el bloque `generate...endgenerate`:

¿cuántas instancias del módulo tag_compare se crean para W=4?

3.2 Ejercicio 6 — Síntesis con Yosys: contar celdas por valor de W

Ejercicio

Paso 1 — Ejecutar la síntesis.

```
cd hw/ && make synth
```

El script `run_sweep.sh` sintetiza `cache_top.v` (comparador + PLRU) para $W \in \{1, 2, 4, 8, 16\}$ y guarda `synth_results.csv`.

Paso 2 — Inspeccionar un log de síntesis y completar:

Yosys reporta el número de celdas por tipo (`$_AND_`, `$_XOR_`, `$_MUX_`, etc.). Completar la tabla con los valores del CSV:

Configuración	W	**Celdas comb.**	**FFs**	**Wire bits**	**Total celdas**
Direct-Mapped	1				
2-Way	2				
4-Way	4				
8-Way	8				
Fully Assoc.	16				

Paso 3 — Observar el tipo de celdas. En el log de síntesis, ¿qué tipo de celda domina para W grande?